Lesson 3 – Sensitive Information Disclosure

A vulnerable admin workflow allowed unauthorized generation of a signed S3 receipt download link through backend code execution.

1. Goal and Vulnerability Summary

The goal of this lesson was to demonstrate that privileged backend receipt functionality could be abused to obtain access to receipt download links stored in Amazon S3 outside the intended authorization boundary. The affected components were the vulnerable backend request-processing path, the administrative Lambda function **DVSA-ADMIN-SHELL**, the receipt-generation Lambda function **DVSA-ADMIN-GET-RECEIPT**, and the receipts S3 bucket. The security impact was unauthorized disclosure of receipt-related data through a generated signed download link. The weakness was caused by unsafe backend code execution combined with weak control around administrative receipt operations.

2. Why This Works / Root Cause

This vulnerability was possible because attacker-controlled input could reach executable backend logic instead of being treated as normal data. In addition, the administrative shell function trusted a user context that could reach privileged code paths and used **eval(cmd)** on input supplied through the request body. Once arbitrary code execution was achieved, the attacker could invoke **DVSA-ADMIN-GET-RECEIPT**, which generated a signed S3 download link for receipt archives. The core problems were unsafe code execution and weak authorization boundaries around receipt-generation functionality.

3. Environment and Setup

DVSA was deployed in **us-east-1** in a controlled non-production AWS account. The API route used in this lesson was **POST /order**, and the administrative route **/admin** was also inspected to identify the backend function mapping. The main AWS components involved were API Gateway, the **DVSA-ORDER-MANAGER** Lambda function, the **DVSA-ADMIN-SHELL** Lambda function, the **DVSA-ADMIN-GET-RECEIPT** Lambda function, the **DVSA-USERS-DB** DynamoDB table, and the receipts S3 bucket. An attacker DVSA account was used to obtain a valid Cognito access token from browser local storage. Postman, AWS Console, CloudWatch Logs, DynamoDB, Lambda, and webhook.site were used during the demonstration.

4. Reproduction Steps

4.1. The API Gateway routes were reviewed, and the **/admin** route was found to map to the **DVSA-ADMIN-SHELL** Lambda function.

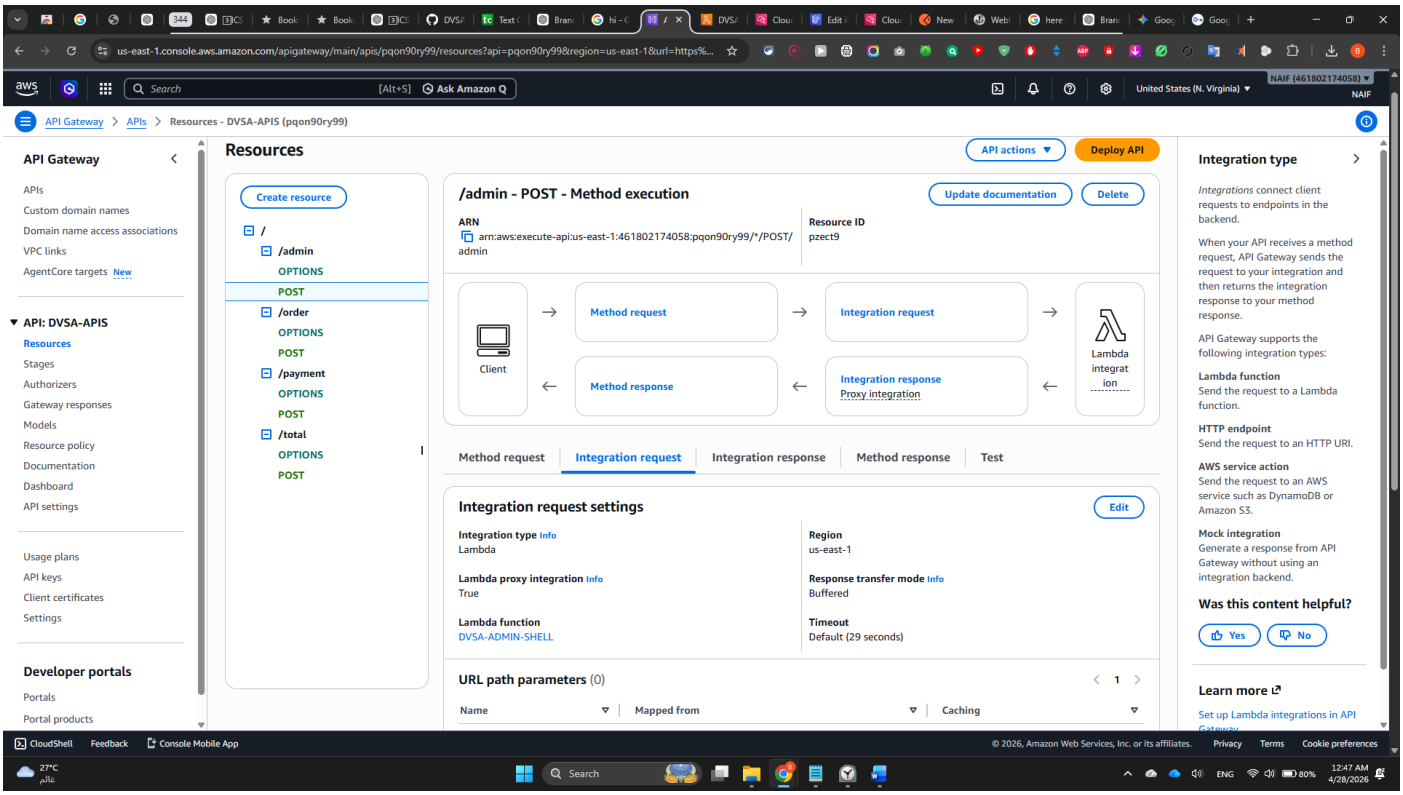

Figure 1. API Gateway /admin integration mapped to DVSA-ADMIN-SHELL.

4.2. The **DVSA-ADMIN-SHELL** function code was inspected, and it was confirmed that the function contained **eval(cmd)**, which allowed execution of attacker-controlled JavaScript code.

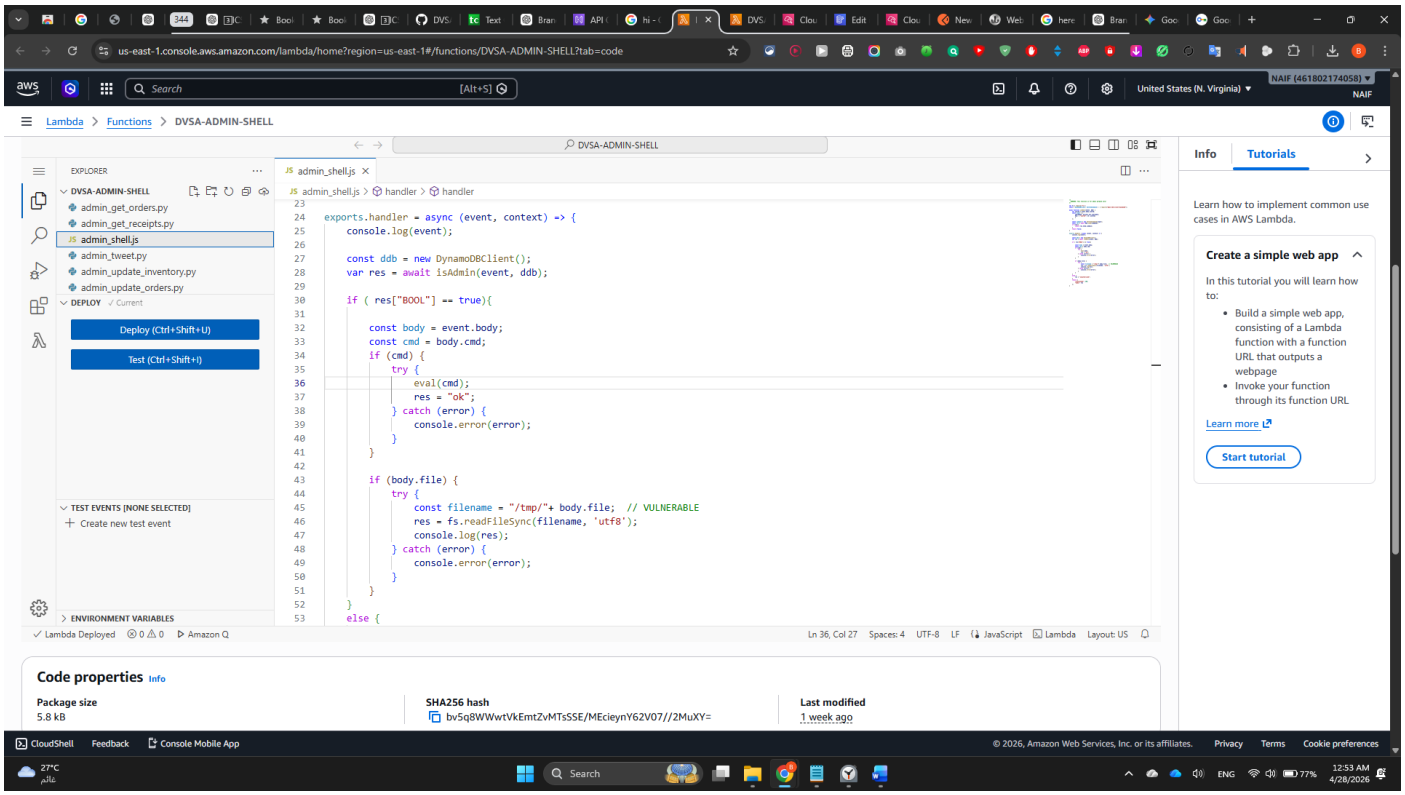

Figure 2. DVSA-ADMIN-SHELL source code showing dangerous eval(cmd) usage.

4.3. The same Lambda code also showed that it checked whether the supplied **userId** belonged to an admin user by reading from the **DVSA-USERS-DB** table.

4.4. The **DVSA-USERS-DB** table was inspected, and an item with **isAdmin = true** was identified.

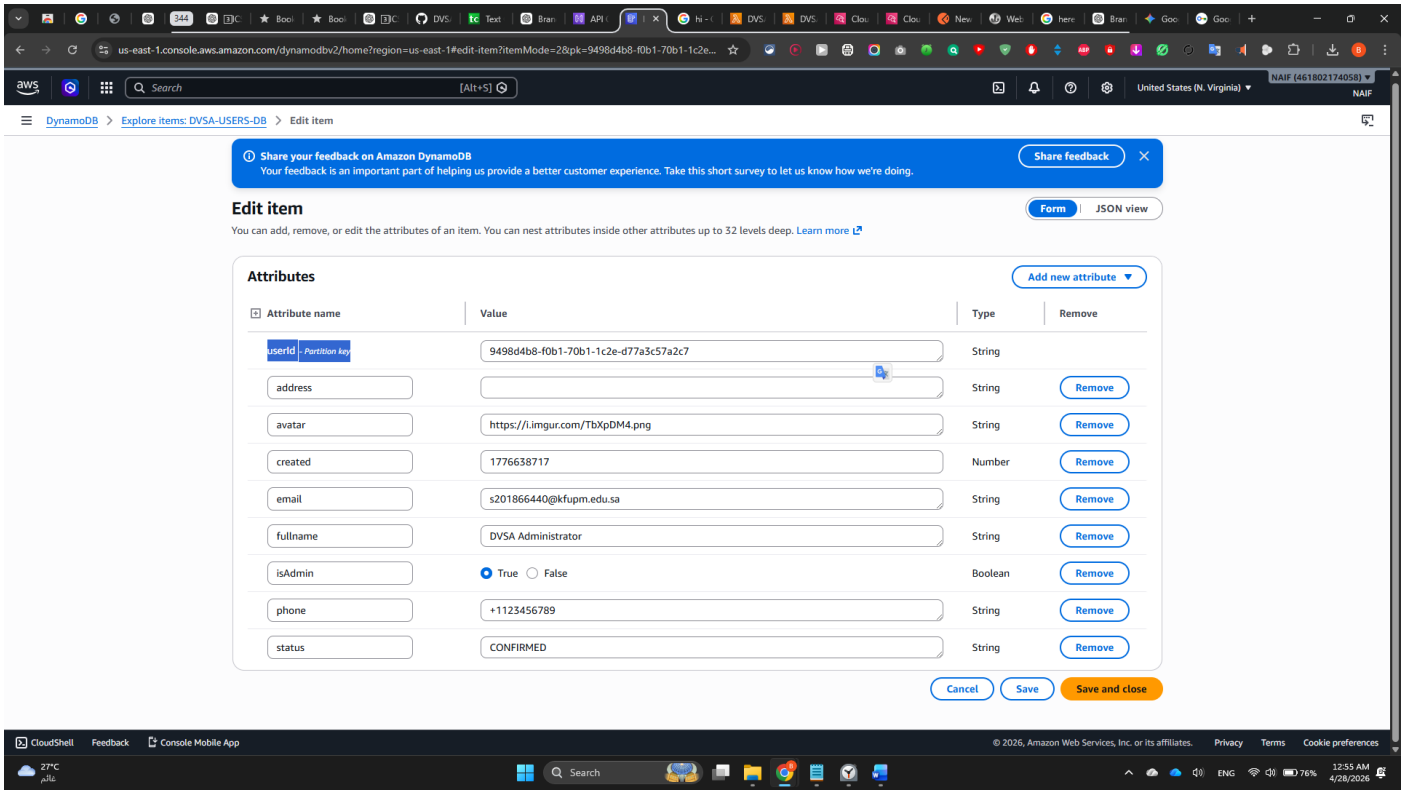

Figure 3. DVSA-USERS-DB item showing an administrative user record with isAdmin = True.

4.5. The attacker logged into the DVSA application with a normal test account and extracted a valid Cognito **accessToken** from browser local storage.

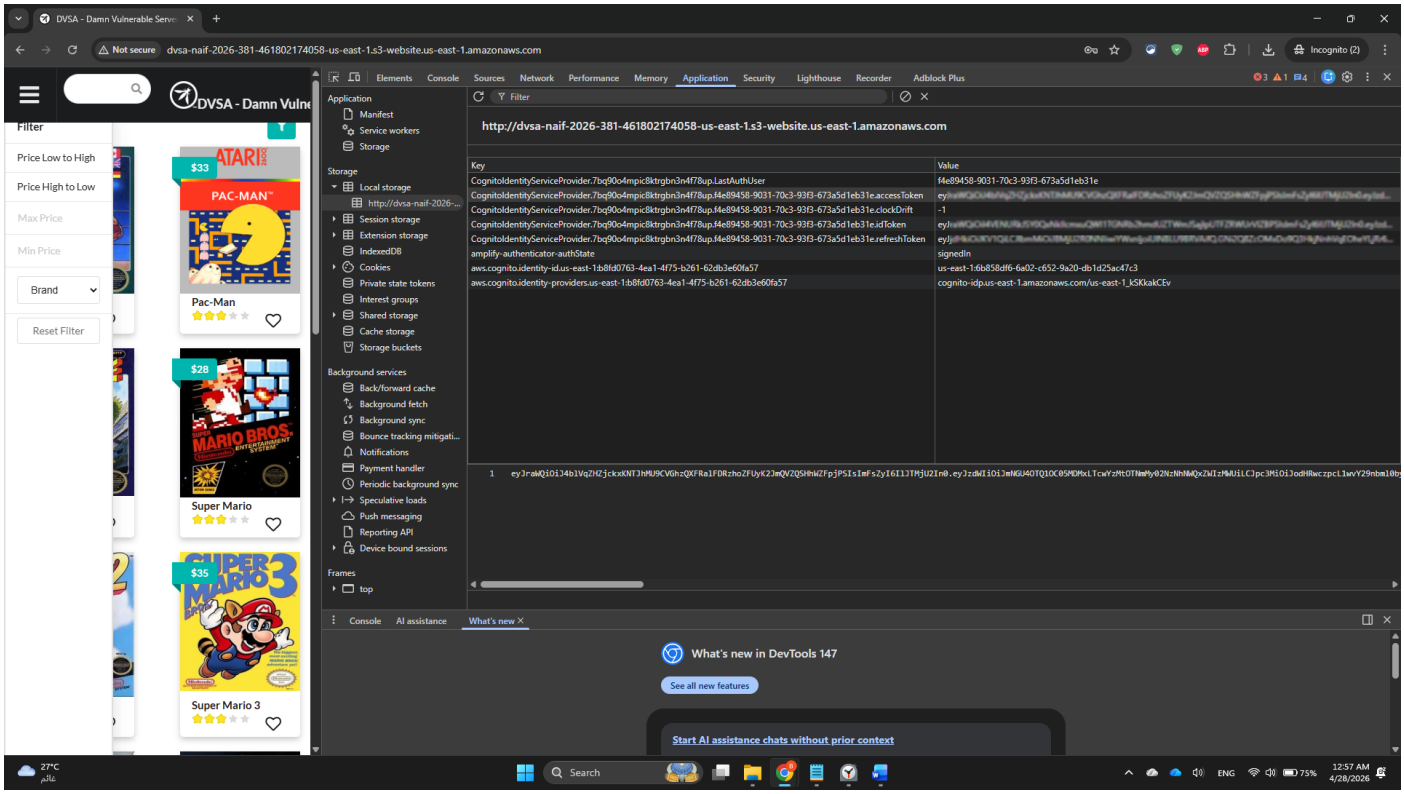

Figure 4. Attacker Cognito access token located in browser local storage (redacted).

4.6. A crafted request was sent to **POST /Stage/order** in Postman with the malicious payload in the request body and the valid attacker **Authorization: Bearer ...** header.

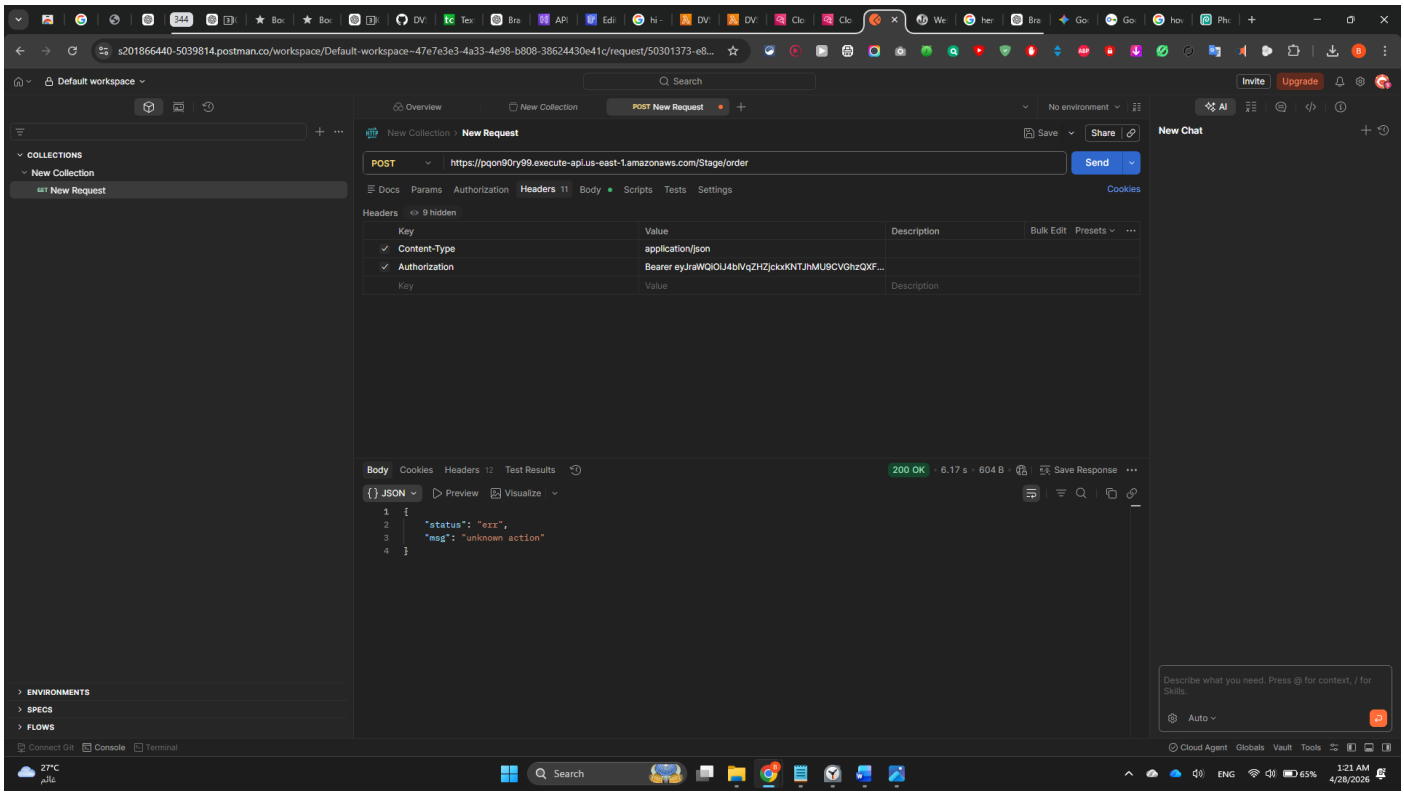

Figure 5. Successful malicious Postman request to POST /Stage/order with authenticated attacker context (redacted).

4.7. The payload invoked the **DVSA-ADMIN-GET-RECEIPT** Lambda function and redirected the returned result to webhook.site.

4.8. The webhook received a request containing **status: ok** and a **download_url**, confirming that the administrative receipt-generation function had been successfully invoked and that a signed receipt download link had been produced.

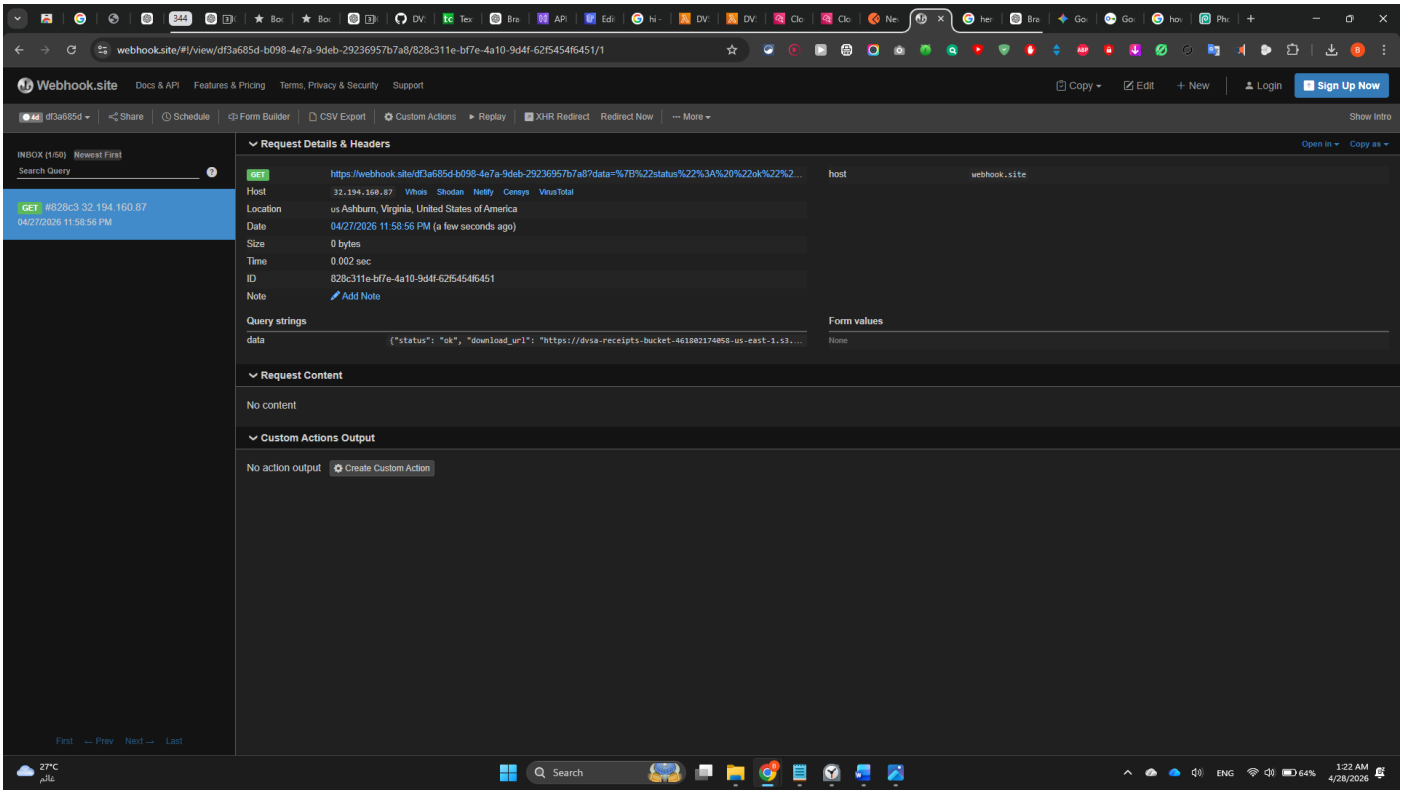

Figure 6. Webhook capture showing status: ok and a signed receipt download_url (redacted).

5. Evidence and Proof

The vulnerability was proven through several pieces of evidence. First, the **/admin** integration showed that administrative requests reached **DVSA-ADMIN-SHELL**. Second, the **DVSA-ADMIN-SHELL** source code clearly contained **eval(cmd)**, which allowed arbitrary code execution if attacker-controlled input reached that field. Third, the DynamoDB table **DVSA-USERS-DB** showed that an admin user record existed, confirming the presence of privileged user logic. Fourth, the attacker extracted a valid Cognito access token from browser local storage and used it in the crafted request. Finally, webhook.site captured the Lambda result, which included **status: ok** and a signed **download_url**. This confirmed that sensitive receipt-related information could be generated and exposed outside the intended authorization boundary.

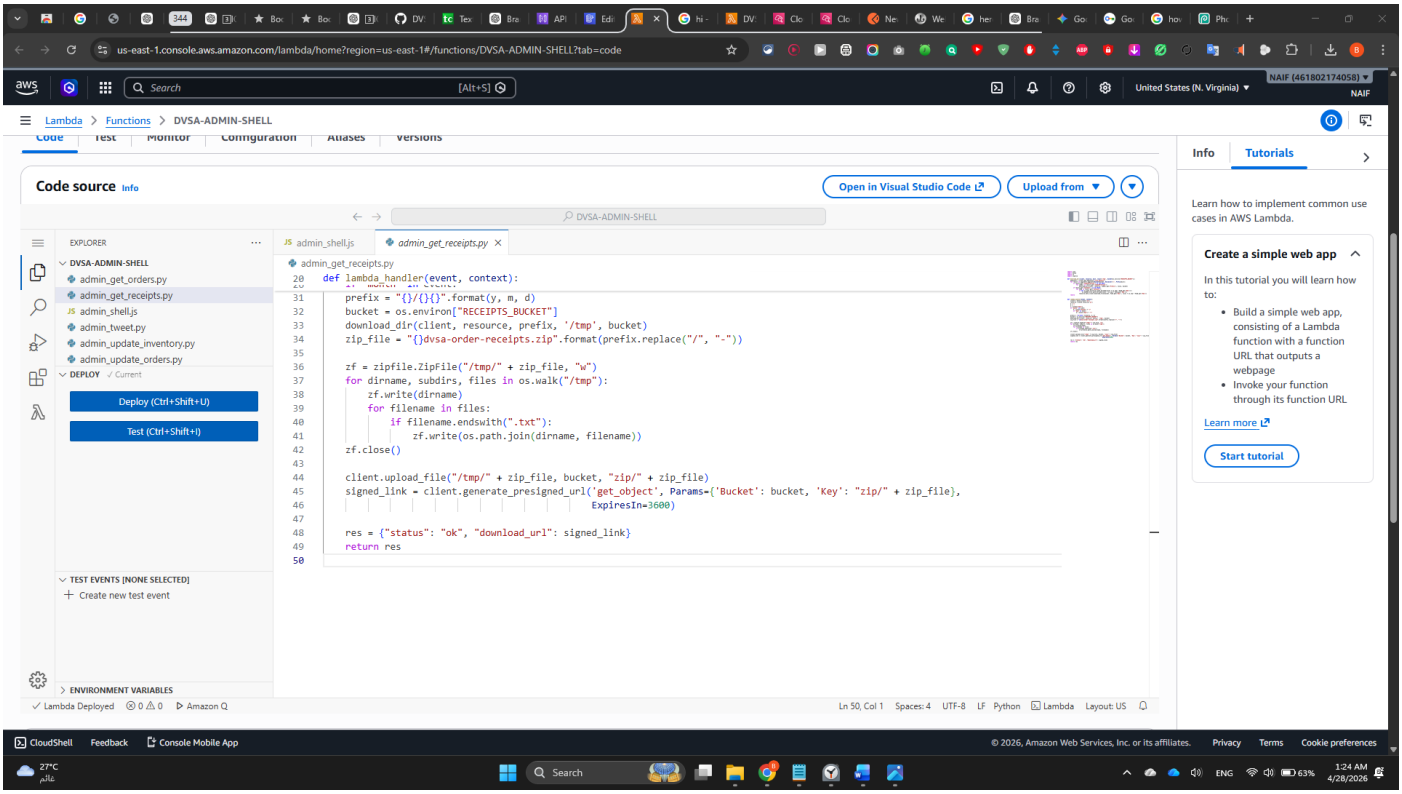

Figure 7. admin_get_receipts.py code generating a signed S3 download link.

6. Fix Strategy / Probable Mitigation

The fix should remove arbitrary code execution from the administrative workflow and enforce strict authorization on receipt-generation functionality. The **DVSA-ADMIN-SHELL** function should not use **eval()** on attacker-controlled input. Instead, it should allow only a limited set of safe backend actions through explicit server-side logic. In addition, receipt-generation operations should require strong authorization checks so that only properly authorized administrative contexts can generate receipt archives or signed S3 download links. The signed URL generation logic should not be reachable from a path that can be influenced by untrusted users.

7. Code / Config Changes

The **DVSA-ADMIN-SHELL** function was updated to safely parse the incoming request body before use. The vulnerable **eval(cmd)** execution path was removed from practical use by blocking unsafe command execution attempts and returning a safe response instead. This change prevented attacker-

controlled input from being executed as backend JavaScript. The administrative logic was therefore constrained to safer server-side behavior and no longer allowed arbitrary code execution through the previous command path.

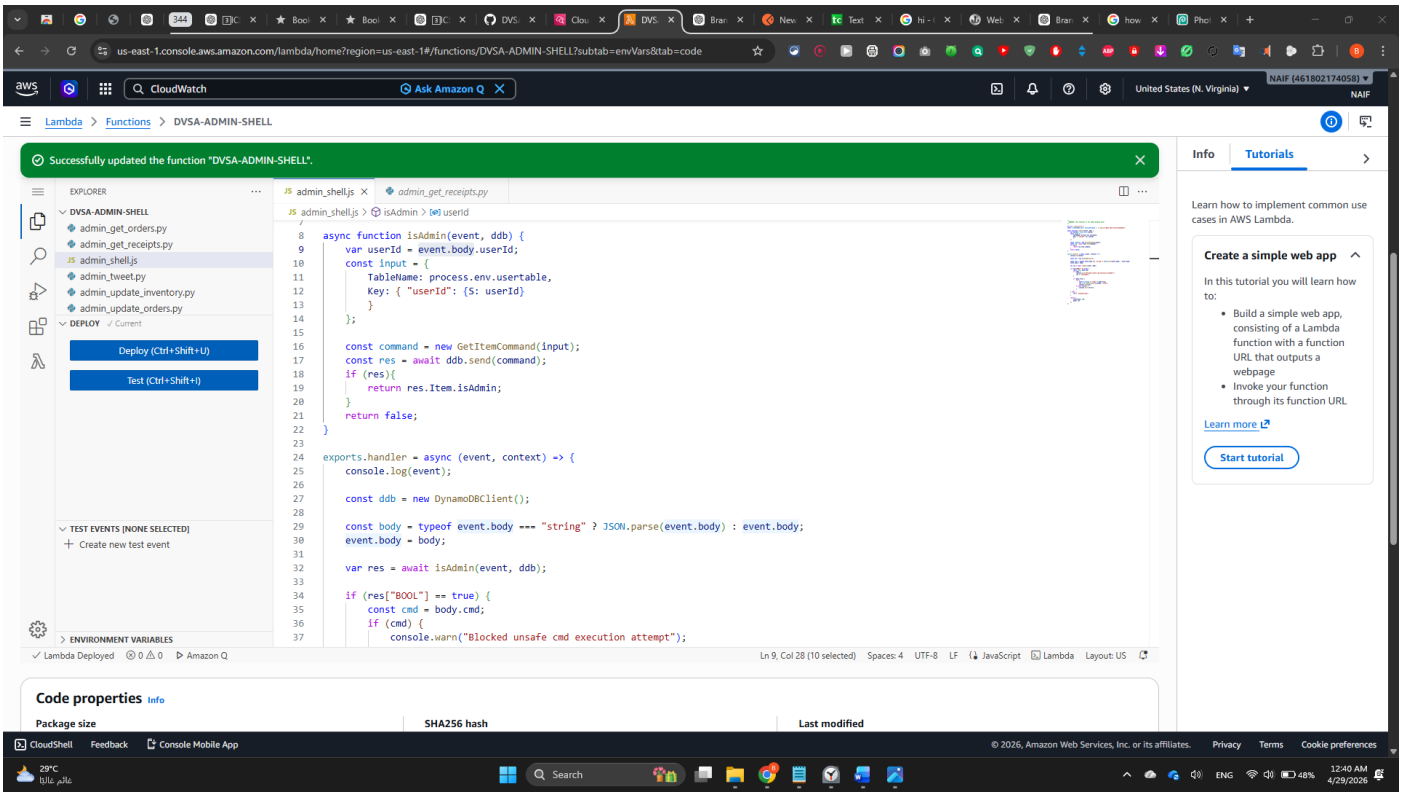

Figure 8. DVSA-ADMIN-SHELL code after remediation, showing request-body parsing and blocked unsafe cmd execution.

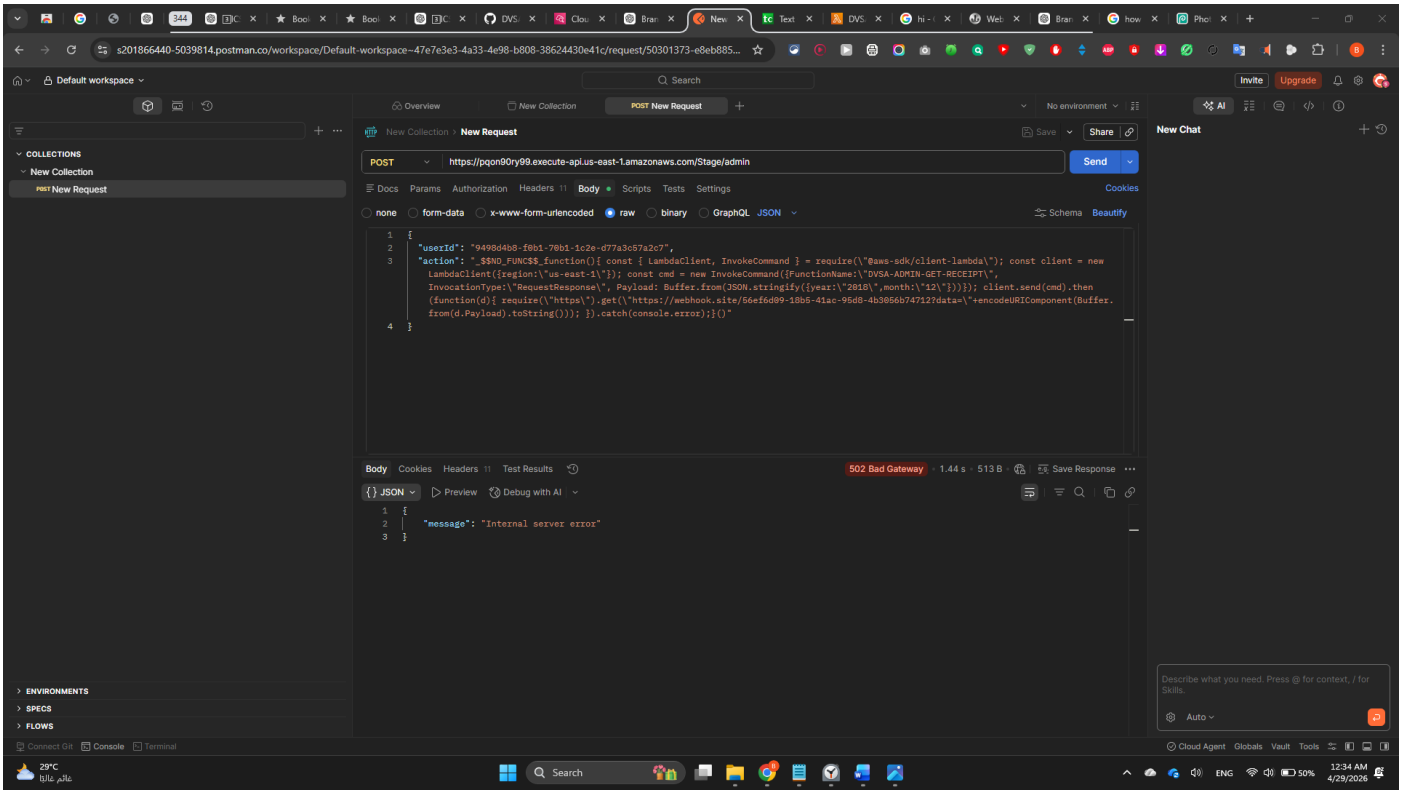

Figure 9. Post-fix request to the /admin route no longer reproduces the original successful exploit behavior.

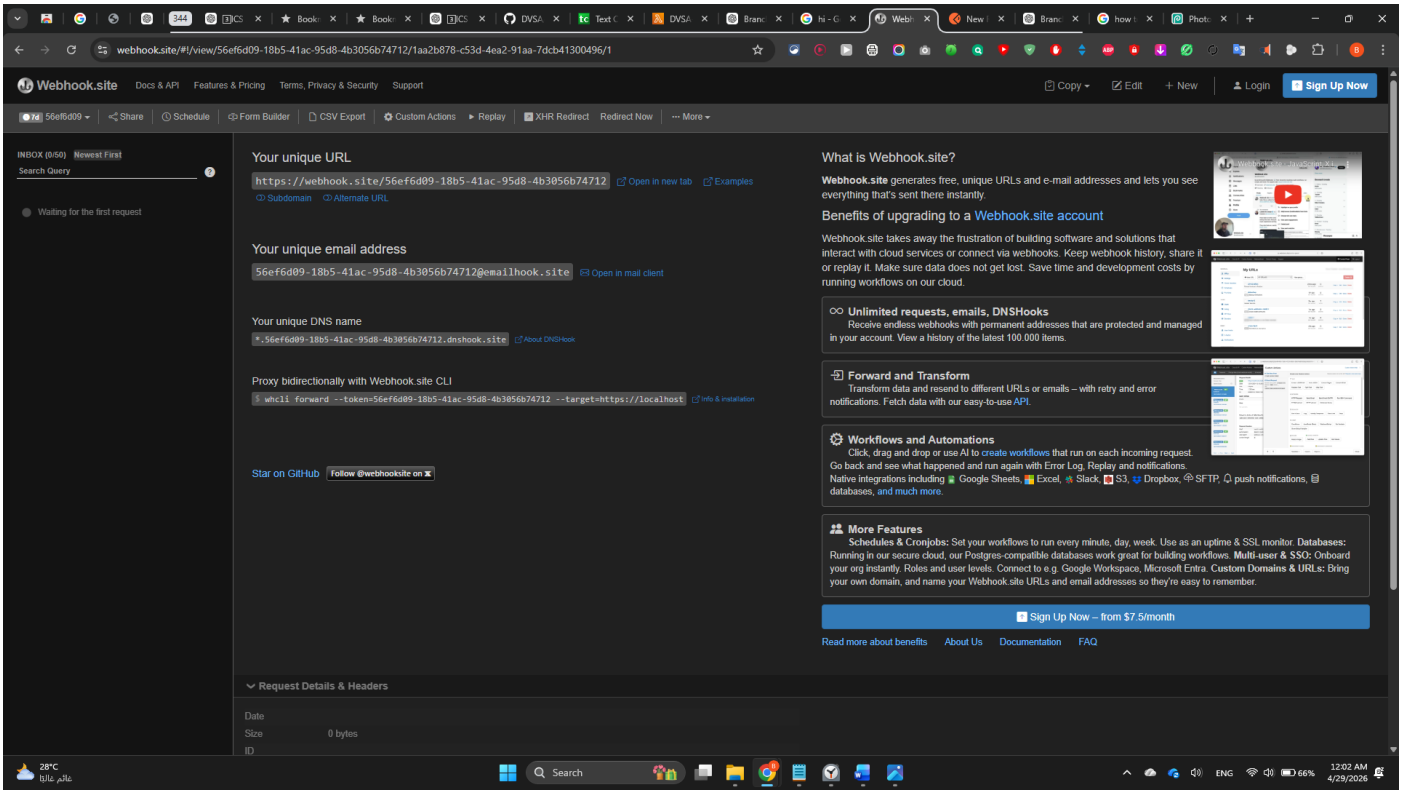

Figure 10. Webhook after remediation showing no valid leaked receipt download result.

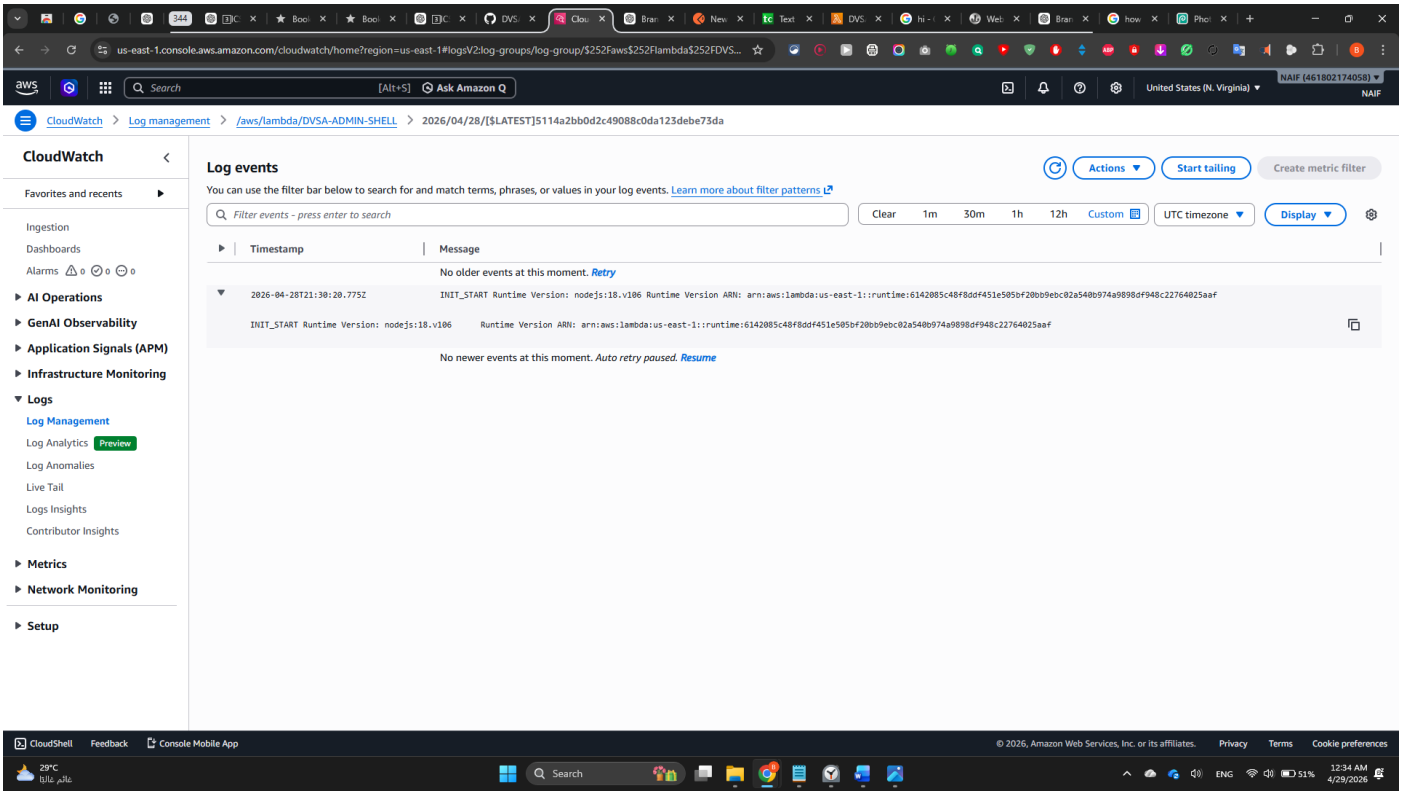

Figure 11. CloudWatch post-fix validation showing the previous exploit path no longer succeeds.

8. Verification After Fix

After remediation, the same malicious request pattern was tested again. A direct request to the administrative route no longer reproduced the original exploit behavior and returned an error response instead of a successful execution path. Webhook inspection did not show a valid exfiltrated receipt download result after the fix. In addition, CloudWatch logs no longer showed the previous successful

exploit path. These checks confirmed that the original arbitrary code execution path used to generate unauthorized signed receipt links was no longer functioning after remediation.

9. Structured Operation and Security Analysis

Intended Logic and Security Rule(s):

The administrative receipt workflow should only allow trusted administrative backend actions. Receipt archive generation and signed S3 download link creation should be restricted to authorized admin operations and must never be reachable through attacker-controlled code execution.

Evidence Sources and Behavior Trace:

The analysis relied on API Gateway route mapping, Lambda source code, DynamoDB admin-user evidence, browser local storage token extraction, Postman request/response behavior, and webhook.site capture of the exfiltrated Lambda result.

Normal vs. Exploit Behavior:

Under normal behavior, a non-admin user should not be able to trigger administrative receipt-generation logic or receive privileged receipt download links. Under exploit conditions, the attacker's crafted request caused backend code execution and successfully invoked the receipt-generation Lambda, which returned a signed S3 download URL to the webhook.

Why This Is a Security Deviation / Fix / Verification:

This is a security deviation because an untrusted input path should never lead to arbitrary code execution or unauthorized access to privileged receipt-generation functionality. The fix removed arbitrary command execution and enforced safer server-side logic. Verification confirmed that the same malicious request no longer succeeded after remediation.

10. Takeaway / Lessons Learned

This lesson shows that sensitive information disclosure in serverless systems can happen when administrative backend functions are reachable through unsafe execution paths. Even if the final data is stored securely in S3, weak authorization boundaries and arbitrary code execution can still expose signed download links and bypass the intended security model. Backend logic should never evaluate attacker-controlled input, and sensitive operations such as receipt generation must be tightly restricted.

Appendix / Runtime Note

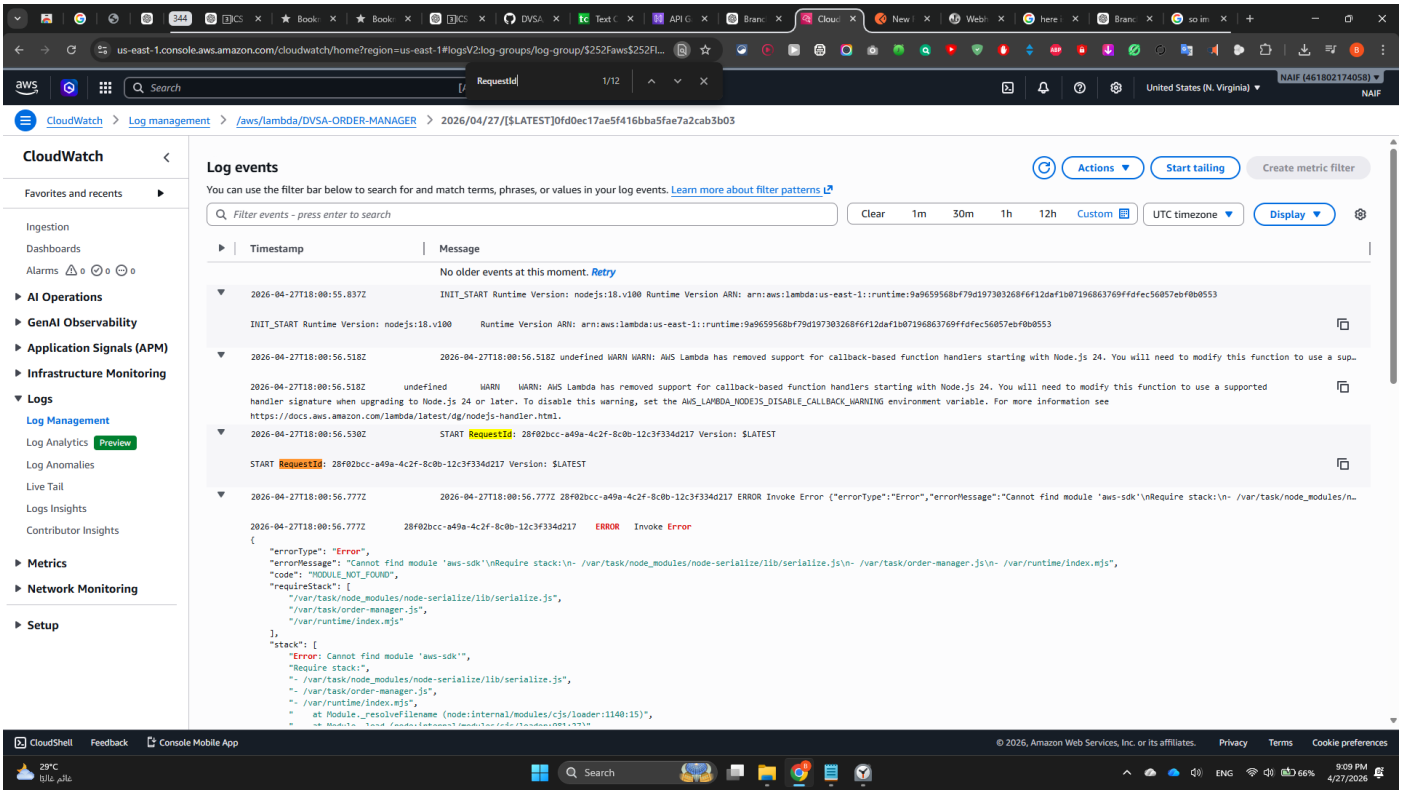

Figure 12. CloudWatch evidence showing the original legacy payload failed due to missing aws-sdk in the current runtime.